

Self-Healing RAG Pipeline

with Drift Detection & Auto-Remediation — built on n8n

Author	Ali Jawwad
Stack	n8n · Postgres + pgvector · FastAPI · Claude · OpenAI Embeddings
Type	Senior-level Agentic AI Automation Project
Status	Roadmap & specification — ready to build

What this document is

A complete project specification you can build from. It explains the problem, the solution architecture, every step of the pipeline in plain language, and a phased 7-week roadmap. Each technical term is defined inline the first time it appears.

CONTENTS

1.	Background — what RAG is, why it degrades, what self-healing means
2.	Pipeline Architecture — the seven cooperating components
3.	Implementation Roadmap — Phase 0 through Phase 5, week by week
4.	Tooling Checklist — what to set up before writing code
5.	Immediate Next Steps — how to start this week

1. Background

What is RAG?

RAG (Retrieval-Augmented Generation) means your AI looks up relevant information from your documents *before* answering, instead of relying purely on its training data. Think of it as an open-book exam — the AI reads your knowledge base, then writes the answer.

The Core Problem

RAG systems silently degrade over time. Documents get edited, embeddings (numerical fingerprints of meaning stored in a vector database) become stale, retrieval quality drops, and answers slowly get worse. Most teams only notice when customers complain — by then, trust is already damaged.

What "Self-Healing" Means

A self-healing pipeline catches its own decline (called **degradation**) and fixes itself automatically. Instead of building a RAG system once and watching it slowly decay, you build one that monitors itself, detects problems, and repairs itself — like a car that books its own service appointments before breaking down.

Why this is a senior-level project

Most engineers can build a basic RAG. Very few can build the eval (evaluation), drift detection, and feedback loops that keep one healthy in production. This project demonstrates ML-ops thinking applied to AI systems — exactly the skillset infrastructure-heavy AI teams hire for.

2. Pipeline Architecture

The system is made up of seven cooperating components. Each one is a sub-workflow inside n8n that can run independently, but together they form a closed feedback loop — outputs from later steps train and improve earlier steps.

Step 1 — Continuous Quality Checking

Every few hours, n8n samples real user queries from the last 24 hours and re-runs them through the RAG. A second AI (Claude) acts as judge and grades each answer for relevance, accuracy, and completeness. This pattern is called **LLM-as-judge** — using one AI to evaluate another AI's work — and is the foundation of all modern AI evaluation systems.

Step 2 — Drift Detection

Drift is the slow mismatch between what an embedding represents and what the underlying content actually means today. A weekly job re-embeds a sample of chunks with the current model and compares them to a saved baseline using **cosine similarity** (a math measure of how similar two fingerprints are, ranging from 0 = unrelated to 1 = identical). If too many chunks have drifted past threshold, they are queued for re-embedding.

Step 3 — Source Change Detection

When someone edits a document in Notion, Sanity, or Google Drive, a **webhook** (an automatic notification sent over HTTP) fires into n8n. The system compares the old and new content using cosine similarity. If the change is meaningful (similarity below 0.95), affected chunks are re-embedded. If it's cosmetic — a typo fix, a date change — the update is ignored to save cost.

Step 4 — Incremental Re-Embedding

Drifted or changed chunks are re-encoded using the latest embedding model. Only affected chunks are updated, never the whole database. This is called **incremental indexing** and is essential at scale — re-embedding millions of chunks every time something changes is both slow and prohibitively expensive.

Step 5 — Investigator Agent

When quality scores drop, an autonomous agent (built with the OpenAI Agents SDK) investigates *why*. It can call diagnostic tools to inspect failed queries, check which chunks were retrieved, test alternative retrieval strategies, and check chunk freshness. Its output is a structured diagnosis: *"67% of failures retrieved stale pricing docs from Q2"* — actionable, not just a metric drop.

Step 6 — Canary Deployments

A **canary deployment** means rolling a change out to a small slice of traffic (typically 5%) before everyone gets it. The name comes from canaries that miners used to detect dangerous gas. When testing a new embedding model or retrieval strategy, n8n routes a fraction of traffic to it and compares eval scores before promoting the change to 100%.

Step 7 — Closed Feedback Loop

Everything the system learns — failed queries, user thumbs-down feedback, agent diagnoses, fixes that worked — flows back into the eval dataset. The pipeline gets smarter every week without manual retraining, because each iteration is informed by real production failures rather than synthetic benchmarks.

3. Implementation Roadmap

Don't build all seven steps at once. Below is the realistic order to ship this in, with each phase designed to be independently valuable. After Phase 2 you already have a portfolio-worthy project.

PHASE 0 · Week 1

Build a Boring RAG First

- **Vector store:** Postgres with the pgvector extension. You already know Postgres — don't add Pinecone or Weaviate complexity yet.
- **Source documents:** Pick ONE source. Your blog, 50 PDFs, a Notion workspace — doesn't matter, just pick one and commit.
- **Embedding model:** OpenAI text-embedding-3-small. Cheap, fast, good enough for v1.
- **Chat endpoint:** A FastAPI route that takes a question, retrieves the top-5 most similar chunks, sends them to Claude with the question, returns the answer.
- **Logging:** Every query, retrieved chunks, final answer, and timestamp written to a Postgres table. This data is what enables every later phase.

Goal: A working RAG endpoint you can hit with curl and get answers. Nothing fancy.

PHASE 1 · Week 2

Add the Eval Loop

Build this entirely in n8n as a single scheduled workflow:

- **Cron node** — runs every 6 hours.
- **Postgres node** — pulls 20 random queries from the last 24 hours.
- **HTTP Request node** — re-runs each query through your RAG endpoint.
- **Claude node (LLM-as-judge)** — scores each answer 1-5 on relevance, accuracy, and completeness, returning structured JSON.
- **Postgres node** — writes scores to an `eval_runs` table.
- **IF node** — if average score drops below 4.0, send a Slack alert with the failing queries attached.

Goal: A dashboard query that shows answer quality over time. When the line dips, you know something is wrong.

Track Document Changes

- **Webhook trigger** in n8n listening for Notion, Sanity, or Google Drive edits. If your CMS doesn't support webhooks, fall back to a polling cron.
- **Diff logic** — compute the embedding of old vs new content and calculate cosine similarity.
- **Branch on similarity:** below 0.95 means meaningful change, so trigger re-embedding for affected chunks. At 0.95 or above, ignore as cosmetic.
- **Test manually** — edit a tracked doc, then verify the chunk's vector and updated_at timestamp changed in Postgres.

Goal: When you edit a doc, your vector store updates automatically within one minute.

Drift Detection

- **Snapshot baseline:** a one-time job that copies current embeddings into a *baseline_embeddings* table.
- **Weekly drift cron:** for a sample of 100 chunks, re-embed with the current model and compare to baseline via cosine similarity.
- **Threshold logic:** if more than 10% of chunks have drifted past your threshold, flag the affected chunks for re-embedding.
- **Trend storage:** store drift scores over time so you can graph them and spot slow degradation before it becomes a crisis.

Goal: A 'drift %' metric you can graph over weeks. The trickiest concept, so save it for after the easy wins.

PHASE 4 · Weeks 5-6

The Investigator Agent

This is where your OpenAI Agents SDK skills come in. Build an agent with these tools:

- **get_failed_queries()** — fetches queries that scored below threshold.
- **inspect_retrieved_chunks(query_id)** — shows what was actually pulled from the vector store.
- **re_query_with_different_k(query, k)** — tries the same query with more or fewer retrieved chunks.
- **check_chunk_freshness(chunk_id)** — shows when the chunk was last updated.

When eval scores drop below threshold, n8n triggers the agent. It reasons through failures and produces a written diagnosis. Output goes into a Linear or Notion ticket with proposed fixes — ready for you to review with your morning coffee.

Goal: A morning report explaining WHY quality dropped, not just that it dropped.

PHASE 5 · Week 7+

Canary Deployments & Feedback Loop

- Add a *model_version* column to the queries table so you can compare performance across versions.
- When testing a new embedding model, n8n routes 5% of traffic to it via an IF node.
- Compare eval scores between versions over a one-week window before promoting.
- Wire user feedback (thumbs up/down on answers) directly into the eval dataset so the system learns from real use.

Goal: A pipeline that gets measurably better every week without manual retraining.

Honest advice: stop after Phase 2

Phases 0 to 2 alone are already a portfolio-worthy project that 95% of self-described "RAG developers" can't actually build. Ship it, write a blog post, post the GitHub repo, then decide if you want to keep going. The classic mistake on ambitious projects is building Phase 5 before Phase 1 actually works. Build it boring, build it observable, then make it smart.

4. Tooling Checklist

Set these up before writing any code:

Component	Recommendation
Workflow engine	Self-hosted n8n on a small VPS (Hetzner \$5/month box) or n8n Cloud trial

Vector database	Postgres with pgvector — Supabase free tier or local Docker Postgres
LLM API (judge & chat)	Anthropic Claude API key
Embedding API	OpenAI API key — text-embedding-3-small
Test interface	Simple Next.js page that hits your RAG endpoint conversationally
Version control	Git repo from day one — export n8n workflows as JSON and commit them

5. Immediate Next Steps

- **This week:** spin up Postgres + pgvector locally, get a basic /chat FastAPI endpoint returning real answers from a small document set.
- **Next week:** wire up the n8n eval loop and watch the first quality scores roll in.
- **Build in public:** tweet/post each phase as you ship it. The OpenAI Agents SDK contributor angle plus a self-healing RAG project is rare positioning.
- **Document failures:** screenshot every interesting eval failure and what fixed it. This becomes your blog post and your interview material.

Why this project matters for your career

Most RAG demos collapse in production. Building one that monitors and repairs itself signals exactly the kind of senior, ML-ops-aware engineering that distinguishes AI infrastructure engineers from prompt-tweakers. Combined with your Agents SDK contributions, this is a strong differentiator.